

Exosphere

Enhanced Encrypted Traffic Analysis

Project Report
Ashwin Anand 2023UCP1644
Vinit Khorwal 2023UCP1683

Introduction

With the rapid growth of internet traffic, VPN tunneling and other protocols such as TLS encrypt packet payloads to protect user privacy and prevent unauthorised access. Although this provides confidentiality it creates significant challenges for cybersecurity systems. Traditional IDSes and Deep Packet Inspection techniques rely on payload visibility but this is not possible in tunneled traffic. However certain metadata remains visible such as:

- Packet Lengths
- Packet inter-arrival times
- Packet ordering
- Flow durations
- Packet burst patterns

These characteristics form distinguishable fingerprints that can reveal patterns similar to attacks.

Exosphere is a CNN-based IDS that utilises packet lengths and inter-arrival times for this purpose.

Objectives

- Reproduce the Exosphere framework results using GitHub repo.
- Understand and complete preprocessing and training pipeline in the original architecture.
- Analyse strengths and limitations of the CNN-based architecture.
- Novelty Work and comparisons.

Contributions

- Successfully reproduced the Exosphere encrypted traffic analysis framework.
- Detailed Analysis of the official Implementation.
- Replacement of CNN-based architecture with a multi-layer LSTM.
- Additional Data pre-processing steps to address class imbalance.
- Comparative Evaluation between CNN and LSTM.
- Experimental Analysis of different LSTM configurations.

Original Framework

Exosphere uses packet inter-arrival times and packet lengths to assume attack behavior.

Main Idea is that different attack categories produce different traffic patterns even when payloads are encrypted.

Converts Raw-Data to a sequence of

- Packet Inter arrival times
- Packet sizes (normalised)

These are then fed to a U-Net inspired Semi-Circular CNN.

Feature and Data representation

The Dataset includes 3 values per sample - Timestamp, Packet length and attack label.

For feature Engineering, there are 3 steps:

- Packet Length Normalisation: Normalised by Ethernet MTU values.
- Inter Arrival Time Calculation: Time Difference is Computed and normalised as: $T(i) - T(i-1) / T(\max)$
 - Intentionally negative to represent that the quantity is time.
- Final Sequence Construction:
 - Alternates between Normalised packet length and inter-arrival times.
 - [-IAT1, LEN1, -IAT2, LEN2 ...]
- Then the Sequence is divided into Segments of size 1000.
- Each segment becomes one training sample.
- Tensor Shape used by CNN (batch, 1, 1000).

Semi-Circular CNN

The original paper uses a U-Net inspired CNN architecture consisting of:

- Convolutional Blocks (1D conv)
- MaxPooling Layers
- UpSampling Layers
- Skip-Connections

CNN architecture is effective because:

- Nearby packets have correlated behaviour.
- Attacks may create localised burst patterns.
- Conv1D layers efficiently learn Short-Range signatures.

Reproduction of the Github Implementation

Environment Setup:

- The official github repo was cloned.
- Python 3.12
- Pytorch nightly build with CUDA 13.0
- RTX 5060 GPU
- Windows Environment.

The Dataset provided in the Github Repo was used.

The Processing pipeline from original repo was preserved to ensure reproducibility.

Results:

Amplification Attacks				
Dataset	Best F1	Best AUC	Best MCC	Best Accuracy
CLDAP	0.9940	0.9996	0.9880	0.9991
DNS	0.9935	0.9999	0.9870	0.9988
Memcached	0.9981	0.9998	0.9961	0.9995
NTP	0.9535	1.0000	0.9110	0.9837

Brute Force Attacks				
Dataset	Best F1	Best AUC	Best MCC	Best Accuracy
ICMP	0.9661	0.9942	0.9328	0.9978
UDP	0.9373	0.9903	0.8749	0.9956
SYN	0.9643	0.9974	0.9291	0.9975
RST	0.9358	0.9978	0.8760	0.9973

Application Attacks				
Dataset	Best F1	Best AUC	Best MCC	Best Accuracy
DNS	0.9899	0.9984	0.9799	0.9983
SMTP	0.9747	0.9971	0.9496	0.9972
RDP	0.9939	0.9999	0.9877	0.9980
NetBIOS	0.9294	0.9967	0.8641	0.9888

Dataset	Best Epoch	Best F1	Best AUC	Best MCC	Best Accuracy
FUZZ	12	0.9668	0.9983	0.9347	0.9962
SSDPDOS	11	0.9845	0.9993	0.9691	0.9984
SYNDOS	11	0.9862	0.9969	0.9725	0.9989
UDPDOS	13	0.9761	0.9902	0.9531	0.9891

These are the Best Scores however there was some model Instability, some logs showed overfitting in later epochs despite good stable learning in earlier ones.

Baseline Analysis

The original model performs extremely well across almost all datasets and scenarios but suffers from overfitting and comparatively poor F1-scores in some attack Scenarios.

Example: in the UDPDOS flooding simulation the model overfits on epoch 13 and falling to an F1 score of just 0.68 from 0.96. (massive losses)

In other attack simulations such as Application attacks, better results can be achieved. Due to CNN only capturing short term patterns.

Hence, the model suffers from overfitting in some scenarios and better performance can be extracted by changing the model architecture and better capturing long term dependencies.

LSTM architecture.

Motivation: Although CNNs are highly effective at capturing localised packet patterns, they are limited to model long-term temporal dependencies.

Encrypted traffic is inherently sequential and often resolves over time, hence contains temporal relationships. These are missed by CNNs.

LSTM networks are specifically designed for sequence modelling and temporal dependency learning. Therefore we used an LSTM architecture to improve:

- Temporal traffic modelling
- Sequence dependency learning
- Long range attack behaviour

Model Architecture

The final proposed architecture used:

- Hidden Size = 64
- Number of Layers = 3
- Dropout = 0.2

The architecture consists of

- Multi-Layer LSTM encoder
- Fully connected output layer
- Sequence-level prediction reconstruction.

Additional Improvements

Other Improvements include:

- Dataset rebalancing
- Evaluating multiple LSTM configurations
- Threshold optimisation

Results:

Amplification Attacks	Best F1	Best AUC	Best MCC	Best Accuracy
CLDAP	0.9922	0.9999	0.9843	0.9973
DNS	0.9826	0.9993	0.9652	0.9928
Memcached	0.9978	1.0000	0.9956	0.9989
NTP	0.9962	0.9999	0.9925	0.9974

Bruteforce Attacks	Best F1	Best AUC	Best MCC	Best Accuracy
ICMP	0.9443	0.9982	0.8923	0.9960
RST	0.8462	0.9728	0.6924	0.9759
SYN	0.9047	0.9894	0.8095	0.9789
UDP	0.9192	0.9636	0.8386	0.9939

Application Attacks	Best F1	Best AUC	Best MCC	Best Accuracy
DNS	0.9808	0.9996	0.9619	0.9965
NetBIOS	0.9695	0.9991	0.9395	0.9943
RDP	0.9833	0.9992	0.9667	0.9893
SMTP	0.9611	0.9990	0.9233	0.9954

Flooding Attacks	Best F1	Best AUC	Best MCC	Best Accuracy
FUZZ	0.9674	0.9975	0.9351	0.9906
SSDP-DOS	0.9927	0.9999	0.9853	0.9981
SYN-DOS	0.9912	0.9999	0.9824	0.9982
UDP-DOS	0.9717	0.9890	0.9444	0.9833

Conclusion

CNN Strengths

CNN performed best when attacks had:

- repetitive packet bursts
- local traffic correlations
- stable protocol structures
- fixed amplification signatures

CNN excels at:

localized feature extraction

LSTM Strengths

LSTM performed best when attacks had:

- temporal evolution
- sequential dependencies
- long-duration attack progression
- timing-sensitive behavior

LSTM excels at:

learning temporal behavior over time